

Binary search (4/5)

```
FUNCTION recursiveBinarySearch(A, x, low, high)
  // Base Case 1: Search space is invalid
  IF low > high THEN
    RETURN -1
  END IF

  mid = low + (high - low) // 2

  // Base Case 2: Target found
  IF A[mid] == x THEN
    RETURN mid
  // Recursive Case 1: Search left subarray
  ELSE IF x < A[mid] THEN
    RETURN recursiveBinarySearch(A, x, low, mid - 1)
  // Recursive Case 2: Search right subarray
  ELSE
    RETURN recursiveBinarySearch(A, x, mid + 1, high)
  END IF
END FUNCTION
```

Page - 29

Recurrence Relation:

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

- Time Complexity: $O(\log n)$ by master method

- Best Case: $O(1)$. The target is the first element.
- Worst Case: $O(\log n)$. The target is the last element or not present. Every element must be inspected.
- Average Case: $\Theta(\log n)$. On average, we expect to check $\sim n/2$ elements.



Divide and Conquer Approach

- Recurrence Relation: $T(n) = 2T\left(\frac{n}{2}\right) + 1$
- According to master method, time complexity is $O(n)$.

```
// Recursive helper function
FUNCTION findMaxMinRecursive(A, low, high)
  // Base case: single element
  IF low == high THEN
    RETURN A[low], A[low] // max = min = single element
  END IF

  // Base case: two elements
  IF high == low + 1 THEN
    IF A[low] > A[high] THEN
      RETURN A[low], A[high]
    ELSE
      RETURN A[high], A[low]
    END IF
  END IF

  // Divide: split the array into two halves
  mid = low + (high - low) // 2

  // Conquer: recursively find max/min of left and right halves
  leftMax, leftMin = findMaxMinRecursive(A, low, mid)
  rightMax, rightMin = findMaxMinRecursive(A, mid + 1, high)

  // Combine: compare results from left and right halves
  overallMax = MAX(leftMax, rightMax)
  overallMin = MIN(leftMin, rightMin)

  RETURN overallMax, overallMin
END FUNCTION
```

Page - 35

Divide and Conquer Approach

- Recurrence Relation: $T(n) = T\left(\frac{n}{2}\right) + 1$
- According to master method, time complexity is $O(\log n)$.

```
FUNCTION powerDivideConquer(a, n)
    // a: base number
    // n: non-negative exponent

    // Base case
    IF n == 0 THEN
        RETURN 1
    END IF

    // Recursive case
    half = powerDivideConquer(a, n // 2) // Integer division

    IF n % 2 == 0 THEN
        // n is even
        RETURN half * half
    ELSE
        // n is odd
        RETURN half * half * a
    END IF
END FUNCTION
```

Page - 42

Bubble Sort (3/3)

```
FUNCTION bubbleSort(A)
    // A: the array to be sorted
    n = length(A)
    FOR i FROM 0 TO n-2 DO
        swapped = false
        FOR j FROM 0 TO n-i-2 DO //Last i elements are already in place
            IF A[j] > A[j+1] THEN
                SWAP(A[j], A[j+1])
                swapped = true
            END IF
        END FOR
        // If no two elements were swapped, the array is sorted.
        IF NOT swapped THEN
            BREAK
        END IF
    END FOR
END FUNCTION
```

- **Time Complexity: $O(n^2)$**

- Worst Case: $O(n^2)$ - The array is in reverse order. We need n passes, each doing ~n comparisons.
- Best Case: $O(n)$ - The array is already sorted. The optimized version (with swapped flag) will only do one pass.
- Average Case: $O(n^2)$

Page - 7



Merge Sort (3/5)

- Recurrence Relation: $T(n) = 2T(\frac{n}{2}) + n$
 - ✓ $2T(\frac{n}{2})$: Time to recursively sort two subarrays of size $n/2$.
 - ✓ n Time to merge the two sorted subarrays.
- Time Complexity: $(n \log n)$ in all cases (worst, average, best), calculated by master method.

Page - 10

```
def merge_sort(A, low, high):
    # Base case: a subarray of length 1 or 0 is already sorted
    if low >= high:
        return

    # Recursive case: divide the array into two halves
    mid = (low + high) // 2

    # Recursively sort the left half
    merge_sort(A, low, mid)

    # Recursively sort the right half
    merge_sort(A, mid + 1, high)

    # Merge the two sorted halves
    merge(A, low, mid, high)

def merge(A, low, mid, high):
    # Create temporary arrays for left and right halves
    left = A[low:mid+1]
    right = A[mid+1:high+1]

    i = j = 0
    k = low

    # Merge the two arrays by comparing elements
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            A[k] = left[i]
            i += 1
        else:
            A[k] = right[j]
            j += 1
        k += 1

    # Copy any remaining elements from left array
    while i < len(left):
        A[k] = left[i]
        i += 1
        k += 1

    # Copy any remaining elements from right array
    while j < len(right):
        A[k] = right[j]
        j += 1
        k += 1
```

Quick Sort (3/5)

Here's the code for quicksort:

```
def quicksort(array):
    if len(array) < 2:
        return array  # Base case: arrays with 0 or 1 element are already "sorted."
    else:
        pivot = array[0]  # Recursive case
        less = [i for i in array[1:] if i <= pivot]  # Sub-array of all the elements less than the pivot
        greater = [i for i in array[1:] if i > pivot]  # Sub-array of all the elements greater than the pivot
        return quicksort(less) + [pivot] + quicksort(greater)

print quicksort([10, 5, 2, 3])
```

Page - 15



Quick Sort (4/5)

- **Best Case Recurrence:** $T(n) = 2T(\frac{n}{2}) + n$

Occurs when the pivot always divides the array into two equal halves

- **Worst Case Recurrence:** $T(n) = T(n-1) + n$

Occurs when the **pivot is always the smallest or largest element**

- **Time Complexity:**

- ✓ Best Case: $\Omega(n \log n)$,
- ✓ Average Case: $\Theta(n \log n)$
- ✓ Worst Case: $O(n^2)$

Page - 16



The naive approach for matrix multiplication

- The naive approach directly implements the mathematical definition of matrix multiplication using three nested loops.

- Time Complexity: $O(n^3)$.

```
FUNCTION naiveMatrixMultiply(A, B, n)
    // A, B: input matrices of size n x n
    // n: dimension of matrices

    LET C be a new n x n matrix initialized to zeros

    FOR i FROM 0 TO n-1 DO
        FOR j FROM 0 TO n-1 DO
            FOR k FROM 0 TO n-1 DO
                C[i][j] = C[i][j] + A[i][k] * B[k][j]
            END FOR
        END FOR
    END FOR

    RETURN C
END FUNCTION
```

Page - 21

The classroom scheduling problem using Brute Force algorithm

- **Classroom Scheduling: "Which Classes to Choose?"**

Decision: Include or exclude each class from the schedule.

Order doesn't matter - we only care about the **set** of classes.

Binary choice per class: Take it or leave it.

- Brute force complexity: $O(2^n)$

Page - 10

```
FUNCTION simple_brute_force(classes):
    best = [] // Will store our best schedule

    // Try including/excluding each class in all possible ways
    FOR include_A in [True, False]:
        FOR include_B in [True, False]:
            FOR include_C in [True, False]:
                // ... repeat for all classes

                current = []
                IF include_A: ADD class_A to current
                IF include_B: ADD class_B to current
                IF include_C: ADD class_C to current
                // ... for all classes

                IF no_overlaps(current):
                    IF length(current) > length(best):
                        best = current

    RETURN best
```

Fractional knapsack problem using brute force algorithm

- **Brute Force:** Guaranteed optimal but exponentially slow

- **Time Complexity:** $O(2^n)$

Page - 19

```
FUNCTION brute_force_knapsack(weights, values, capacity):
    n = length(weights)
    max_value = 0

    # Generate all possible subsets (2^n possibilities)
    FOR bitmask FROM 0 TO (2^n - 1):
        total_weight = 0
        total_value = 0

        # Check which items are included in this subset
        FOR i FROM 0 TO n-1:
            IF bitmask has i-th bit set:
                total_weight += weights[i]
                total_value += values[i]

        # If valid and better, update best solution
        IF total_weight <= capacity AND total_value > max_value:
            max_value = total_value

    RETURN max_value
```

Page - 15

```

def classroom_schedule(classes): #  $O(n \log n)$ 
    classes.sort(key=lambda x: x[1])

    selected = []
    last_end = 0

    for start, end in classes:
        if start >= last_end:
            selected.append((start, end))
            last_end = end

    return selected

# Example
classes = [(1, 3), (2, 4), (3, 5), (0, 6), (5, 7), (8, 9)]
print(classroom_schedule(classes))

def job_sequencing(jobs): #  $O(n^2)$ 
    jobs.sort(key=lambda x: x[1], reverse=True)

    max_deadline = max(j[2] for j in jobs)
    schedule = [None] * (max_deadline + 1)
    total_profit = 0

    for job_id, profit, deadline in jobs:
        for slot in range(deadline, 0, -1):
            if schedule[slot] is None:
                schedule[slot] = job_id
                total_profit += profit
                break

    return schedule, total_profit

# Example
jobs = [('A', 100, 2), ('B', 19, 1), ('C', 27, 2), ('D', 25, 1), ('E', 15, 3)]
print(job_sequencing(jobs))

def fractional_knapsack(items, capacity): #  $O(n \log n)$ 
    items = [(v, w, v/w) for v, w in items]
    items.sort(key=lambda x: x[2], reverse=True)

    total_value = 0
    for value, weight, ratio in items:
        if capacity == 0:
            break
        if weight <= capacity:
            total_value += value
            capacity -= weight
        else:
            total_value += ratio * capacity

```

```
        capacity = 0

    return total_value

# Example
items = [(60, 10), (100, 20), (120, 30)]
capacity = 50
print(fractional_knapsack(items, capacity))
```

